Pilgrim Optimizer CLI: First Commands

Purpose

This guide explains the first three CLI commands in ‘pilgrim-optimizer‘ and what they currently prove in the Ruleset A mancala sandbox.

The goal is to make the early loop explicit:

‘scenario -> validate -> legal actions -> search -> recommendation‘

Prerequisites

- Python 3.12+ installed (‘python3 --version‘)
- Project dependencies installed from repository root:

python3 -m pip install -e ".[dev]"

- You are in the repository root:

cd /path/to/pilgrim-optimizer

Command 1: Validate a scenario

python3 -m pilgrim.cli validate scenarios/mancala_sandbox_001.json

What it does right now:

- Loads the scenario JSON file.
- Parses it and loads the linked setup/config JSON files.
- Checks simplified mancala-sandbox invariants.
- Confirms the scenario is a valid starting state for the current engine.

What it does **not** do:

- It does not solve the game.
- It does not prove full Pilgrim rules are implemented.

Typical output:

Scenario ’mancala_sandbox_001’ is valid.

Command 2: List legal actions

python3 -m pilgrim.cli legal-actions scenarios/mancala_sandbox_001.json

What it does right now:

- Loads the same scenario.
- Asks the current rules engine to generate legal **full-turn** actions.

- Each listed action is one complete simplified turn: sow + selected duty/tithe resolution.
- Prints a numbered list of readable full-turn summaries.
- Prints a final legal-action count.

Why this matters:

- It is one of the fastest ways to debug action generation.
- It should usually be checked before trusting solver output.

Example output:

Legal actions for scenario 'mancala_sandbox_001':

1. Turn: sow city -> north -> north_east -> east | selected duty: north | action: produce
2. Turn: sow city -> north -> north_east -> east | selected duty: north | action: tithe
...

Total legal actions: N

Command 3: Run the simple solver

python3 -m pilgrim.cli solve scenarios/mancala_sandbox_001.json --depth 3

What it does right now:

- Runs the current exact-search prototype.
- Searches to the specified depth in **full turns** ('--depth 3' means 3 complete turns).
- Uses a temporary placeholder evaluation function.

Example output:

Solve result for scenario 'mancala_sandbox_001'
Depth: 3
Best score: 3
Nodes expanded: 27

Best first full turn:
Turn: sow city -> north -> north_east -> east | selected duty: east | action: clerical_silversmith

Best line:
1. Turn: sow city -> north -> north_east -> east | selected duty: east | action: clerical_silversmith
2. Turn: sow north -> north_east | selected duty: north_east | action: clerical_devotion
3. Turn: sow city -> south | selected duty: south | action: tithe

How to interpret the current output

- The old machine-oriented token 'best_action=sow:0:1->2->3' is now shown in readable

form.
- ‘city -> north -> north_east -> east‘ corresponds to position IDs ‘0 -> 1 -> 2 -> 3‘.
- ‘nodes_expanded‘ means the number of search nodes explored.
- ‘best_score‘ is still a value under a **temporary placeholder sandbox evaluation**, not true final Pilgrim VP.
- ‘best line‘ is now a sequence of full turns, not alternating sow/resolve sub-actions.
- Together, this confirms the current development loop works end-to-end: ‘scenario -> validate -> legal actions -> search -> recommendation‘.

Using ‘--verbose‘ with ‘solve‘ prints:

- all transition events for the recommended first full turn (sowing + duty/tithe + invariants)
- a compact state summary after applying that first full turn
- ‘Acted player‘ (the player who executed that recommended turn)
- ‘Next active player‘ (the player whose turn is next)
- the acted player state so resource gains and acolyte recall are directly visible
- ‘Piety position‘ and ‘Piety track VP‘ for direct track-value inspection
- an evaluation breakdown for the acted player after the first full turn

Position mapping used by the current sandbox:

- ‘0 = city‘
- ‘1 = north‘
- ‘2 = north_east‘
- ‘3 = east‘
- ‘4 = south_east‘
- ‘5 = south‘
- ‘6 = south_west‘
- ‘7 = west‘
- ‘8 = north_west‘

What this does not mean yet

- The engine is not complete for full Pilgrim gameplay.
- The solver score is not final-game scoring quality.
- This is not strategic proof or balance validation.
- Only the current deterministic mancala-sandbox slice is covered.

Piety Track Scoring (v0.2)

- Piety is now treated as a capped track position (‘max_position = 12‘).
- Piety position and piety VP are different values.
- The VP lookup table is loaded from ‘configs/piety.json‘.
- Sandbox solver evaluation is still temporary, but now uses **piety track VP** instead of raw piety position.

Typical development workflow

1. Edit rules/config/scenario files.

2. Run ‘validate‘ to ensure scenario integrity.
3. Run ‘legal-actions‘ to inspect generated action space.
4. Run ‘solve‘ with a small depth to sanity-check transitions/search loop.
5. Run ‘pytest‘ to protect behavior with tests.

Troubleshooting

- ‘zsh: command not found: python‘
  - Use ‘python3‘ instead of ‘python‘.
  - Optional alias:
    ‘echo ’alias python=python3’ >> ~/.zshrc && source ~/.zshrc‘
- Scenario path errors
  - Confirm file exists: ‘scenarios/mancala_sandbox_001.json‘.
  - Run commands from repository root.
- Validation fails
  - Check JSON syntax and current invariants (acolyte conservation, non-negative resources, legal route lengths).
- Solver output seems surprising
  - Inspect ‘legal-actions‘ first.
  - Remember the current scoring function is a temporary placeholder.
  - Re-run with verbose mode for transition details:
    ‘python3 -m pilgrim.cli solve scenarios/mancala_sandbox_001.json --depth 3 --verbose‘

Next planned CLI improvements

- Clearer validation diagnostics (which invariant failed and why).
- Optional JSON output mode for tools/integration.
- Scenario diff helpers for fast debugging.
- Replay/event export commands for transition traces.
- Additional solver options beyond exact depth-limited search.